Лабораторная работа № 4. Параллельное умножение матриц с использованием OpenMP

1. Описание работы

На языке C реализованы пять вариантов умножения матриц C = A·B. Последовательные версии ijk_seq и ikj_seq отличаются только порядком вложенных циклов, а их параллельные версии ijk_omp и ikj_omp с помощью директивы #pragma omp parallel for делят между потоками строки матрицы C. В блочной версии blocked_omp матрица C разбивается на блоки, и каждый поток вычисляет свои блоки целиком. Время выполнения измерялось функцией omp_get_wtime() как медиана из трёх запусков, а результат каждого варианта сравнивался с ijk_seq с допустимой относительной погрешностью 10^{-9}; все проверки пройдены. Эксперименты проводились на процессоре Intel Core i5-12450H (4 производительных и 4 энергоэффективных ядра, 12 логических потоков, кеш L1 данных 48 КБ и L2 1,25 МБ на производительное ядро, общий L3 12 МБ) в WSL Ubuntu 24.04 с компилятором GCC 13.3.0 и ключами -O3 -fopenmp.

2. Результаты экспериментов

Таблица 1. Влияние размера матрицы (8 потоков, b=32)

Размер матрицы N	Время ijk_seq, с	Время ijk_omp, с	Время ikj_seq, с	Время ikj_omp, с	Время blocked_omp, с
128	0.001363	0.015377	0.000468	0.009708	0.007637
256	0.010566	0.004763	0.003737	0.000959	0.000922
512	0.274627	0.045544	0.025847	0.006984	0.007410
1024	7.110305	0.548197	0.422596	0.087321	0.080222

Таблица 2. Влияние числа потоков (N=1024, b=32)

Потоки	Время ijk_omp, с	S ijk_omp	E ijk_omp	Время ikj_omp, с	S ikj_omp	E ikj_omp	Время blocked_omp, с
1	7.503271	0.95	0.95	0.429891	0.98	0.98	0.249689
2	2.037797	3.49	1.74	0.250698	1.69	0.84	0.122732
4	0.897154	7.93	1.98	0.136620	3.09	0.77	0.080547
8	0.548197	12.97	1.62	0.086663	4.88	0.61	0.076260
16	0.517864	13.73	0.86	0.073471	5.75	0.36	0.065781

Таблица 3. Влияние размера блока (N = 1024, 8 потоков)

Размер блока	Время blocked_omp, с	Отношение $T_{ikj_seq}/T_{blocked_omp}$
8	0.152449	2.77
16	0.095895	4.41
32	0.084329	5.01
64	0.071722	5.89
128	0.074031	5.71

3. Вывод

Быстрее всего работали ikj_omp и blocked_omp: при N = 1024 на 8 потоках они умножили матрицы примерно за 0.08 с, а ijk_seq – за 7.1 с. Варианты ijk и ikj делают одни и те же вычисления, но в ijk матрица B читается по столбцу, то есть с прыжками по памяти, и нужных данных почти никогда нет в кеше. В ikj матрицы B и C читаются по строкам подряд, поэтому кеш используется хорошо, и ikj_seq быстрее ijk_seq почти в 3 раза при N = 256 и в 17 раз при N = 1024. С ростом числа потоков сильнее всех ускорился ijk_omp – в 13 раз на 8 потоках. Это больше числа потоков, потому что у каждого ядра свой кеш, и чем больше ядер работает, тем больше данных в него помещается, но даже так ijk_omp в 6 раз медленнее ikj_omp. Ускорение ikj_omp и blocked_omp после 4–8 потоков почти перестаёт расти: эти версии считают быстро и упираются в скорость оперативной памяти, общей для всех ядер, а 16 потоков больше, чем 12 потоков процессора. Лучшим размером блока оказался 64: такие блоки помещаются в кеш L2, а при b = 8 программа работала вдвое медленнее, так как много времени уходило на сами циклы. На маленьких матрицах (N = 128) параллельные версии оказались медленнее последовательных, потому что запуск потоков занимает больше времени, чем само умножение.